```cpp
/*
 * Proyecto: Solucion_Pregunta02_Examen02_2023_2
 * Archivo:  main.cpp
 * Autor:    J. Miguel Guanira E.
 *
 * Created on 23 de noviembre de 2023, 07:41 PM
 */

#include <iostream>
#include <iomanip>
using namespace std;
#include "FuncionesAuxiliares.h"

int main(int argc, char** argv) {
    //para probar los reportes se muestran en cout
    crearArchivoArticulos("Articulos.csv","Articulos.bin");
    pruebaArchivoArticulos("Articulos.bin");

    crearArchivoVendedores("Vendedores.csv","Vendedores.bin");
    reporteVendedores("Vendedores.bin");

    completarVendedosres("Ventas.txt","Articulos.bin","Vendedores.bin");

    pruebaArchivoArticulos("Articulos.bin");
    reporteVendedores("Vendedores.bin");

    return 0;
}

/*
 * Proyecto: Solucion_Pregunta02_Examen02_2023_2
 * Archivo:  Articulo.h
 * Autor:    J. Miguel Guanira E.
 *
 * Created on 23 de noviembre de 2023, 07:43 PM
 */

#ifndef ARTICULO_H
#define ARTICULO_H

struct Articulo{
    char codigo[8];
    char descripcion[60];
    int stock;
    double precio;
    int cantidadVedida;
    double montoVendido;
    int cantidadNOVedida;
    double montoPerdido;
};

#endif /* ARTICULO_H */

/*
 * Proyecto: Solucion_Pregunta02_Examen02_2023_2
 * Archivo:  ArticuloPedido.h
 * Autor:    J. Miguel Guanira E.
 *
 * Created on 23 de noviembre de 2023, 07:49 PM
 */

#ifndef ARTICULOPEDIDO_H
#define ARTICULOPEDIDO_H

struct ArticuloPedido{
    char codigo[8];
    int cantidad;
    int fecha;
};
```

```
70
71  #endif /* ARTICULOPEDIDO_H */
72
73  /*
74   * Proyecto: Solucion_Pregunta02_Examen02_2023_2
75   * Archivo:  Vendedor.h
76   * Autor:    J. Miguel Guanira E.
77   *
78   * Created on 23 de noviembre de 2023, 07:51 PM
79   */
80
81  #ifndef VENDEDOR_H
82  #define VENDEDOR_H
83  #include "ArticuloPedido.h"
84  struct Vendedor{
85      int codigo;
86      char nombre[60];
87      double porcentaje;
88      double cuotaMinima;
89      struct ArticuloPedido articulosVendidos[30];
90      int cantidadArtVen;
91      struct ArticuloPedido articulosNOVendidos[30];
92      int cantidadArtNOVen;
93      double montoVendido;
94      double montoNoVendido;
95  };
96
97  #endif /* VENDEDOR_H */
98
99
100 /*
101  * Proyecto: Solucion_Pregunta02_Examen02_2023_2
102  * Archivo:  FuncionesAuxiliares.h
103  * Autor:    J. Miguel Guanira E.
104  *
105  * Created on 23 de noviembre de 2023, 07:56 PM
106  */
107
108 #ifndef FUNCIONESAUXILIARES_H
109 #define FUNCIONESAUXILIARES_H
110
111 void crearArchivoArticulos(const char *,const char *);
112 void pruebaArchivoArticulos(const char *);
113 void crearArchivoVendedores(const char *,const char *);
114 void reporteVendedores(const char *);
115 void completarVendedosres(const char *,const char *, const char *);
116 void completarArchivos(char *,int,int,int,fstream &, fstream &);
117 int cantidaDeRegistros(fstream &,int);
118 int buscarArticulo(char *,fstream &,int );
119 int buscarVendedor(int ,fstream &,int );
120 void modificarArticulo(struct Articulo &,int ,double &, bool &);
121 void modificarVendedor(struct Vendedor &,char*,int,double,int,bool);
122 void agregarVenta(char *,int,int,struct ArticuloPedido *,int &);
123 void reporteDeVentas(const char *, struct ArticuloPedido *,int,double);
124
125 #endif /* FUNCIONESAUXILIARES_H */
126
127 /*
128  * Proyecto: Solucion_Pregunta02_Examen02_2023_2
129  * Archivo:  FuncionesAuxiliares.cpp
130  * Autor:    J. Miguel Guanira E.
131  *
132  * Created on 23 de noviembre de 2023, 07:56 PM
133  */
134
135 #include <iostream>
136 #include <fstream>
137 #include <iomanip>
138 using namespace std;
```

```cpp
#include <cstring>
#include "Articulo.h"
#include "Vendedor.h"
#include "FuncionesAuxiliares.h"
#define NO_ENCONTRADO -1

void crearArchivoArticulos(const char *nombArchTxt,const char *nombArchBin){
    ifstream archTxt(nombArchTxt,ios::in);
    if(not archTxt.is_open()){
        cout<<"ERROR: No se pudo abrir el archivo "<<nombArchTxt<<endl;
        exit(1);
    }
    ofstream archBin(nombArchBin,ios::out|ios::binary);
    if(not archBin.is_open()){
        cout<<"ERROR: No se pudo abrir el archivo "<<nombArchBin<<endl;
        exit(1);
    }
    struct Articulo articulo{};
    char codigo[8], descripcion[60],c;
    int stock;
    double precio;
    while(true){
        archTxt.getline(codigo,8,',');
        if(archTxt.eof())break;
        archTxt.getline(descripcion,60,',');
        archTxt>>stock>>c>>precio;
        archTxt.get();
        strcpy(articulo.codigo,codigo);
        strcpy(articulo.descripcion,descripcion);
        articulo.stock = stock;
        articulo.precio=precio;
        archBin.write(reinterpret_cast<const char*>(&articulo),
                                                    sizeof(struct Articulo));
    }
}

void pruebaArchivoArticulos(const char *nombArchBin){
    ifstream archBin(nombArchBin,ios::in|ios::binary);
    if(not archBin.is_open()){
     cout<<"ERROR: No se pudo abrir el archivo "<<nombArchBin<<endl;
     exit(1);
    }
    struct Articulo articulo;
    cout.precision(2);
    cout<<fixed;
    while(true){
        archBin.read(reinterpret_cast<char*>(&articulo),
                                                    sizeof(struct Articulo));
        if(archBin.eof())break;
        cout<<left<<setw(10)<<articulo.codigo<<setw(55)<<articulo.descripcion
            <<right<<setw(5)<<articulo.stock<<setw(10)<<articulo.precio<<endl;
    }
}

void crearArchivoVendedores(const char *nombArchTxt,const char *nombArchBin){
    ifstream archTxt(nombArchTxt,ios::in);
    if(not archTxt.is_open()){
        cout<<"ERROR: No se pudo abrir el archivo "<<nombArchTxt<<endl;
        exit(1);
    }
    ofstream archBin(nombArchBin,ios::out|ios::binary);
    if(not archBin.is_open()){
        cout<<"ERROR: No se pudo abrir el archivo "<<nombArchBin<<endl;
        exit(1);
    }
    struct Vendedor vendedor{};
    int codigo;
    char nombre[60], c;
    double porcentaje,cuotaMinima;
```

```
208       while(true){
209           archTxt>>codigo;
210           if(archTxt.eof())break;
211           archTxt.get();
212           archTxt.getline(nombre,60,',');
213           archTxt>>porcentaje>>c>>cuotaMinima;
214           vendedor.codigo = codigo;
215           strcpy(vendedor.nombre,nombre);
216           vendedor.porcentaje = porcentaje;
217           vendedor.cuotaMinima = cuotaMinima;
218           archBin.write(reinterpret_cast<const char*>(&vendedor),
219                                                       sizeof(struct Vendedor));
220       }
221   }
222
223   void reporteVendedores(const char *nombArchBin){
224       ifstream archBin(nombArchBin,ios::in|ios::binary);
225       if(not archBin.is_open()){
226        cout<<"ERROR: No se pudo abrir el archivo "<<nombArchBin<<endl;
227        exit(1);
228       }
229       struct Vendedor vendedor;
230       cout.precision(2);
231       cout<<fixed;
232       while(true){
233           archBin.read(reinterpret_cast<char*>(&vendedor),
234                                                       sizeof(struct Vendedor));
235           if(archBin.eof())break;
236           cout<<left<<setw(10)<<vendedor.codigo<<setw(40)<<vendedor.nombre
237               <<right<<setw(5)<<vendedor.porcentaje
238               <<setw(10)<<vendedor.cuotaMinima<<endl;
239           reporteDeVentas("VENDIDOS",vendedor.articulosVendidos,
240                           vendedor.cantidadArtVen,vendedor.montoVendido);
241           reporteDeVentas("NO VENDIDOS",vendedor.articulosNOVendidos,
242                           vendedor.cantidadArtNOVen, vendedor.montoNoVendido);
243       }
244    }
245
246   void reporteDeVentas(const char *titulo, struct ArticuloPedido *articulos,
247                        int cantidad, double monto){
248       cout<<titulo<<endl;
249       for(int i=0; i<cantidad; i++)
250           cout<<left<<setw(10)<<articulos[i].codigo
251                   <<right<<setw(10)<<articulos[i].cantidad
252                   <<setw(10)<<articulos[i].fecha<<endl;
253   }
254
255   void completarVendedosres(const char *nombArchTxt,const char *nombArchArt,
256                             const char *nombArchVen){
257        ifstream archTxt(nombArchTxt,ios::in);
258        if(not archTxt.is_open()){
259            cout<<"ERROR: No se pudo abrir el archivo "<<nombArchTxt<<endl;
260            exit(1);
261        }
262
263        fstream archArt(nombArchArt,ios::in|ios::out|ios::binary);
264        if(not archArt.is_open()){
265            cout<<"ERROR: No se pudo abrir el archivo "<<nombArchArt<<endl;
266            exit(1);
267        }
268
269        fstream archVen(nombArchVen,ios::in|ios::out|ios::binary);
270        if(not archVen.is_open()){
271            cout<<"ERROR: No se pudo abrir el archivo "<<nombArchVen<<endl;
272            exit(1);
273        }
274        char codArt[8],c;
275        int codVen, cant, dd,mm,aa, fecha;
276        while(true){
```

```
277             archTxt>>codArt;
278             if(archTxt.eof())break;
279             archTxt>>codVen>>cant>>dd>>c>>mm>>c>>aa;
280             archTxt.get(); //Saco el cambio de línea
281             fecha = aa*10000+mm*100+dd;
282             completarArchivos(codArt, codVen,cant,fecha, archArt,archVen);
283         }
284     }
285
286     void completarArchivos(char *codArt, int codVen,int cant, int fecha,
287                            fstream &archArt, fstream &archVen){
288         int posA, posV, numRegA, numRegV, tamRegA, tamRegV;
289         double precio;
290         bool vendido;
291         struct Articulo articulo;
292         struct Vendedor vendedor;
293         tamRegA = sizeof(struct Articulo);
294         tamRegV = sizeof(struct Vendedor);
295         numRegA = cantidaDeRegistros(archArt,tamRegA);
296         numRegV = cantidaDeRegistros(archVen,tamRegV);
297         posA = buscarArticulo(codArt,archArt,numRegA);
298         posV = buscarVendedor(codVen,archVen,numRegV);
299         if (posA != NO_ENCONTRADO and posV != NO_ENCONTRADO){
300             archArt.seekg(posA*tamRegA,ios::beg);
301             archVen.seekg(posV*tamRegV,ios::beg);
302             archArt.read(reinterpret_cast<char*>(&articulo),tamRegA);
303             archVen.read(reinterpret_cast<char*>(&vendedor),tamRegV);
304             modificarArticulo(articulo,cant,precio,vendido);
305             modificarVendedor(vendedor,codArt,cant,precio,fecha,vendido);
306             archArt.seekg(posA*tamRegA,ios::beg);
307             archVen.seekg(posV*tamRegV,ios::beg);
308             archArt.write(reinterpret_cast<const char*>(&articulo),tamRegA);
309             archVen.write(reinterpret_cast<const char*>(&vendedor),tamRegV);
310             archArt.flush();
311             archVen.flush();
312         }
313     }
314
315     int cantidaDeRegistros(fstream &arch,int tamReg){
316         int tamArch,numReg;
317
318         arch.seekg(0,ios::end);
319         tamArch = arch.tellg();
320         numReg = tamArch/tamReg;
321         arch.seekg(0,ios::beg);
322         return numReg;
323     }
324
325     int buscarArticulo(char *codigo,fstream &archArt,int numReg){
326         struct Articulo articulo;
327         archArt.seekg(0,ios::beg);
328         for(int i=0; i<numReg; i++){
329             archArt.read(reinterpret_cast<char*>(&articulo),
330                                 sizeof(struct Articulo));
331             if(strcmp(articulo.codigo,codigo)==0)return i;
332         }
333         return NO_ENCONTRADO;
334     }
335
336     int buscarVendedor(int codigo,fstream &archVen,int numReg){
337         struct Vendedor vendedor;
338         archVen.seekg(0,ios::beg);
339         for(int i=0; i<numReg; i++){
340             archVen.read(reinterpret_cast<char*>(&vendedor),
341                             sizeof(struct Vendedor));
342             if(vendedor.codigo == codigo) return i;
343         }
344         return NO_ENCONTRADO;
345     }
```

```
346
347  void modificarArticulo(struct Articulo &articulo,int cant,double &precio,
348                         bool &vendido){
349      if(articulo.stock >= cant){
350          articulo.stock -= cant;
351          articulo.cantidadVedida++;
352          articulo.montoVendido += articulo.precio*cant;
353          vendido = true;
354      }
355      else {
356          articulo.cantidadNOVedida ++;
357          articulo.montoPerdido += articulo.precio*cant;
358          vendido = false;
359      }
360      precio = articulo.precio;
361  }
362
363  void modificarVendedor(struct Vendedor &vendedor,char *codArt,int cant,
364                         double precio, int fecha, bool vendido){
365      if(vendido){
366          vendedor.montoVendido += precio*cant;
367          agregarVenta(codArt, cant, fecha, vendedor.articulosVendidos,
368                       vendedor.cantidadArtVen);
369      }
370      else {
371          vendedor.montoNoVendido += precio*cant;
372          agregarVenta(codArt, cant, fecha,vendedor.articulosNOVendidos,
373                       vendedor.cantidadArtNOVen);
374      }
375  }
376
377  void agregarVenta(char *codArt,int cant,int fecha,
378                    struct ArticuloPedido *artPed,int &cantArtPe){
379      for(int i=0; i<cantArtPe; i++){
380          if(artPed[i].codigo==codArt){
381              artPed[i].cantidad+=cant;
382              if(artPed[i].fecha<fecha)artPed[i].fecha=fecha;
383              return;
384          }
385      }
386      strcpy(artPed[cantArtPe].codigo,codArt);
387      artPed[cantArtPe].cantidad = cant;
388      artPed[cantArtPe].fecha = fecha;
389      cantArtPe++;
390  }
391
392
```